Non-Comparative Sorting

Part 10

1

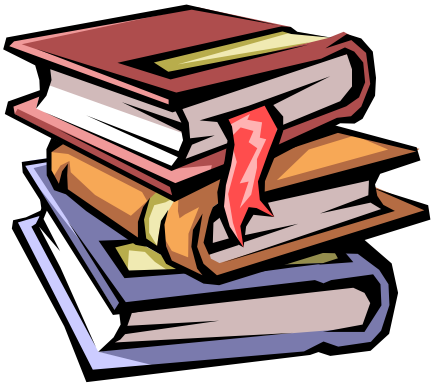
Dictionaries

the "ADT"… not in a human one!

2

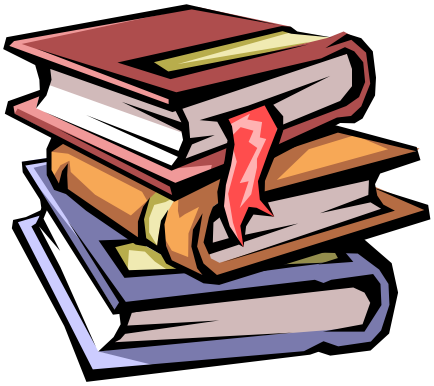
Moving Past Arrays....

- A *collection* is general term for a group of data items
- So, this can include arrays, linked lists, stacks, queues, and much more
- So far, we have just used arrays – which are indexed by an integer



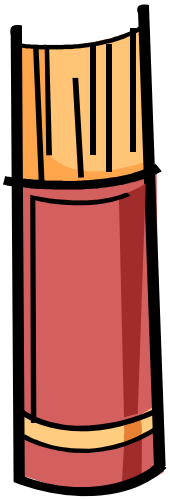
Moving Past Arrays....

- Are there are other ways to index data?
- Yes.
  - any object can be used as an index
  - e.g. strings, integers, pictures, etc...



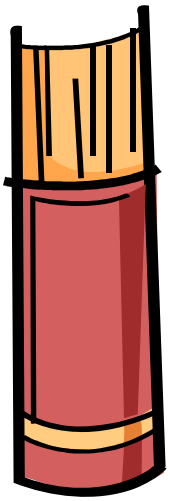
Dictionaries

- Collections of objects indexed by other objects are called *dictionaries*
- They have a few alternative names...
  - keyed tables
  - symbol tables
  - maps



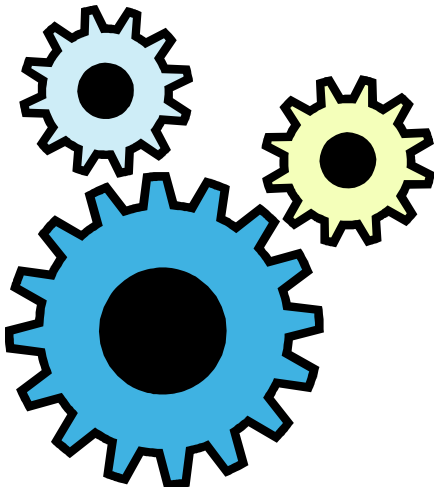
Dictionary Terminology

- The objects that are used for indices are called *keys*
- The objects that are accessed using the key are called *values*



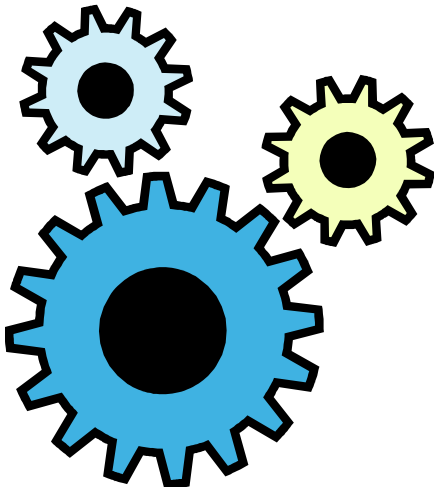
Implementing Dictionaries

- There are numerous approaches to implementing dictionaries
- *Key-value* structure
  - a class stores a key object and value object
  - this can be stored in any data structure we have covered



Implementing Dictionaries

- Using a linked list
  - adding takes O(1)
  - access is O(n)
- Unsorted array
  - add is O(n) – have to resize
  - access is O(n)
- Sorted array
  - add is O(n) – have to resize
  - access is **O(log n)**



This Ain't So Good

- So, adding in to an array is O(n)
- Arrays seem like a poor approach
- Is there a better way to store dictionary data? Keeping adding close to O(1)?
- … and keep access at O(log n)
- *Perhaps, we will learn that soon….*

Databases vs. Dictionaries

- Dictionaries...
  - have a single key
  - that key is the only way to access data
  - key returns a single value
- Databases...
  - may have multiple keys (e.g. SSN, name, age, etc…)
  - may return multiple values

Bucket Sort

The key to sorting is the keys

11

Bucket Sort

- The *Bucket Sort* is a fast sorting algorithm that is non-comparative.
- Rather than comparing objects, it uses mathematical properties of their keys

12

Bucket Sort

- The most basic algorithm creates a "bucket" for each of the different key values
- This "bucket" often takes the form of a queue or list
- Each item in the array is placed into the buckets based on their key

Bucket Sort

- Then, each bucket is emptied, in order, back into the array
- This sorts the items, but algorithm has considerable storage requirements – *often making it impactable*

Bucket Sort – Creating Bucks

```
for (b = minKey; b <= maxKey; b++)
    bucket[b] = new Queue()
end for
```

There is a bucket for each key value. Each bucket is a queue.

Bucket Sort – Scatter Into Buckets

```
for (n = 0; n < array.count; n++)
    bucket[array[n].key].enqueue(array[n])
end for
```

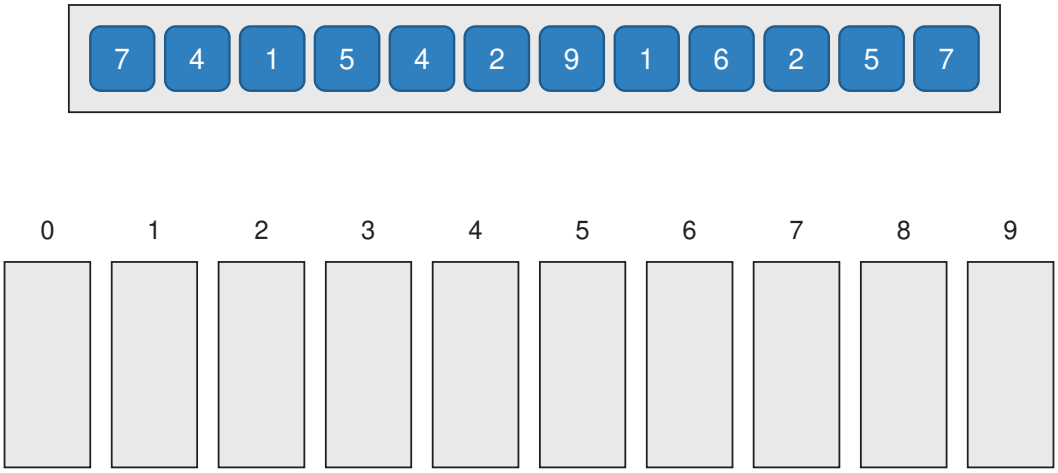
Each array item is put into a bucket based on its key.

Bucket Sort – Gather Back Into Array

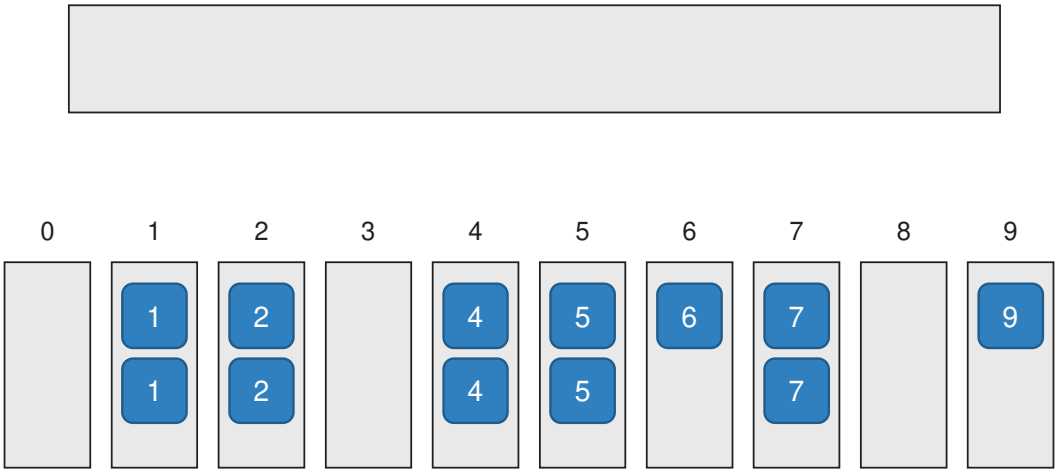
```
n = 0;
for (b = minKey; b <= maxKey; b++)
    while bucket[b] has items
        array[n] = bucket[b].dequeue()
        n++
    end while
end for
```

Empty buckets, in order, to array

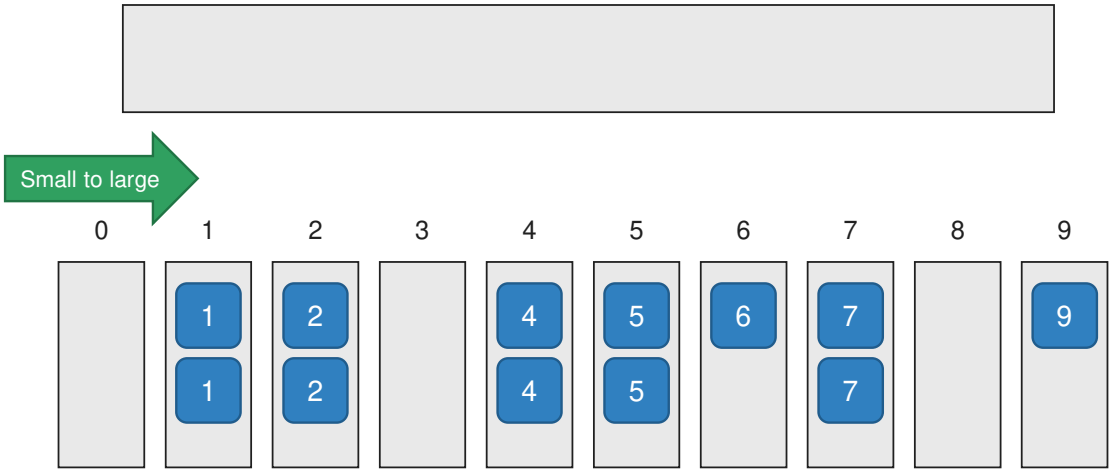
Bucket Sort: Scatter to Buckets



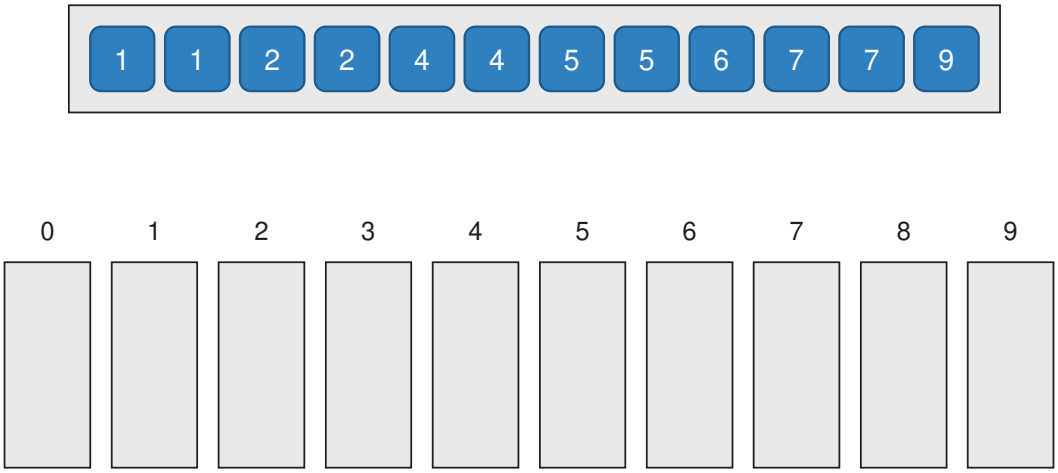
Bucket Sort: Buckets Filled



Bucket Sort: Gather Into Array



Bucket Sort: Complete



Too Many Buckets!

- If we use a bucket for each key, the number of buckets can be **huge!**
- e.g. 32-bit key requires 4,294,967,296 buckets
- So, we need to choose buckets will accept multiple keys <u>within a range</u>

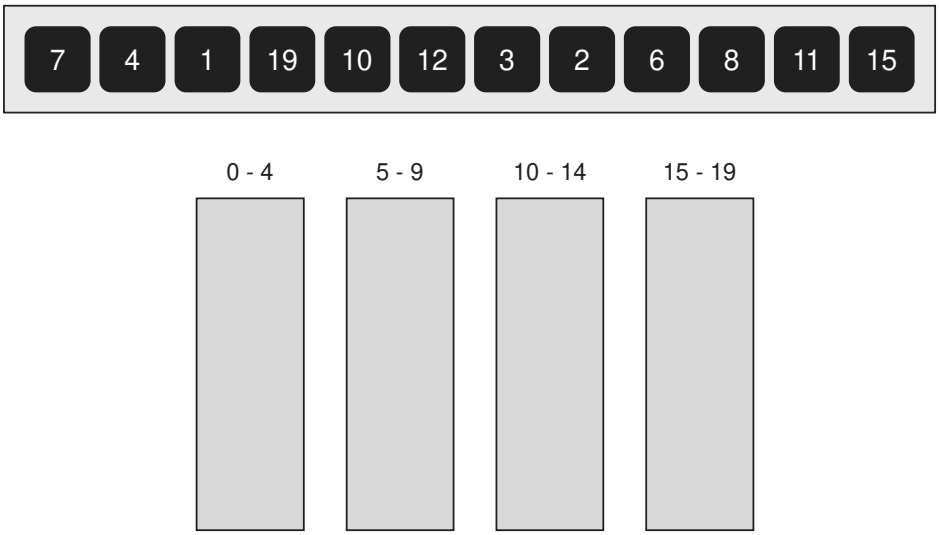
Too Many Buckets

- Naturally, these buckets will contain unsorted keys
- So, we can sort the bucket once it is full
- … then empty the sorted buckets back into the array

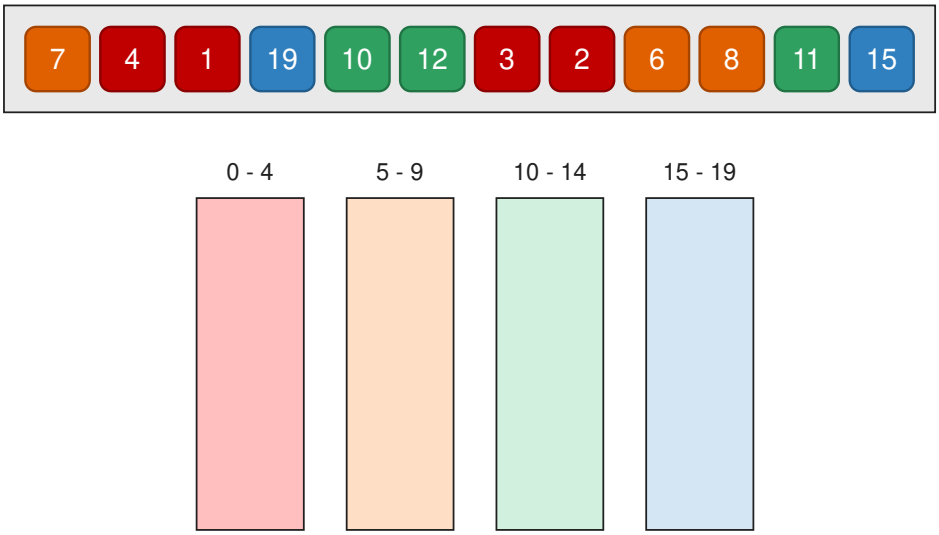
A Better Approach….

1. Fill each bucket with a range of keys
2. Sort each bucket
3. Empty the buckets, in order, into the array

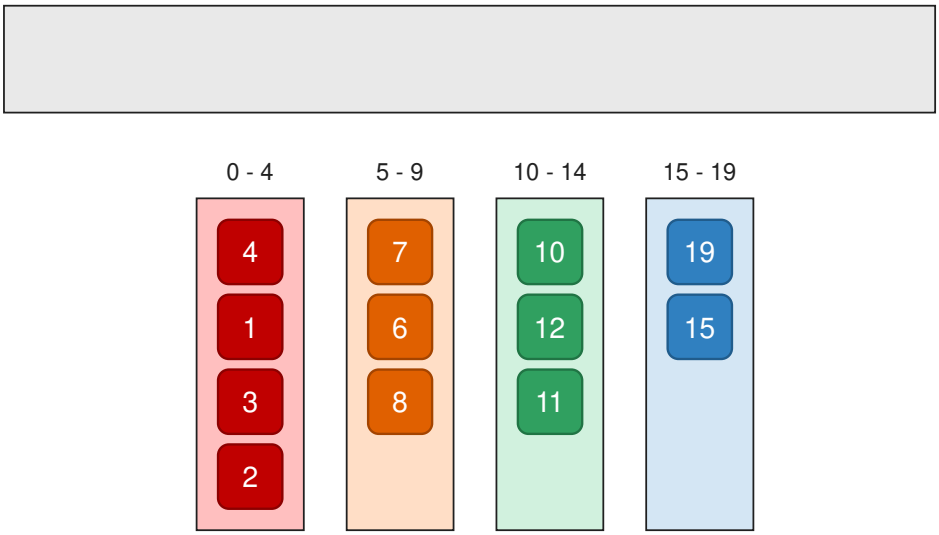
Bucket Sort: Fill Buckets



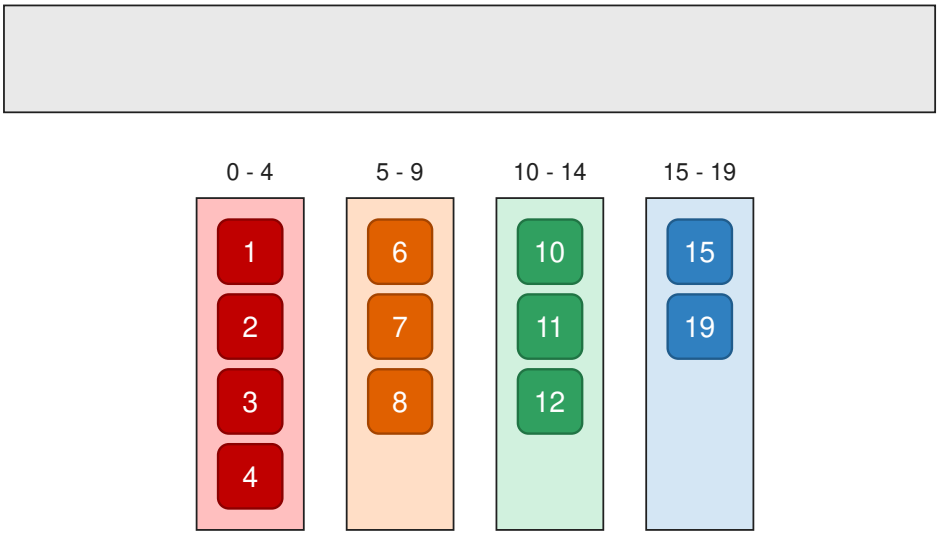
Bucket Sort: Fill Buckets



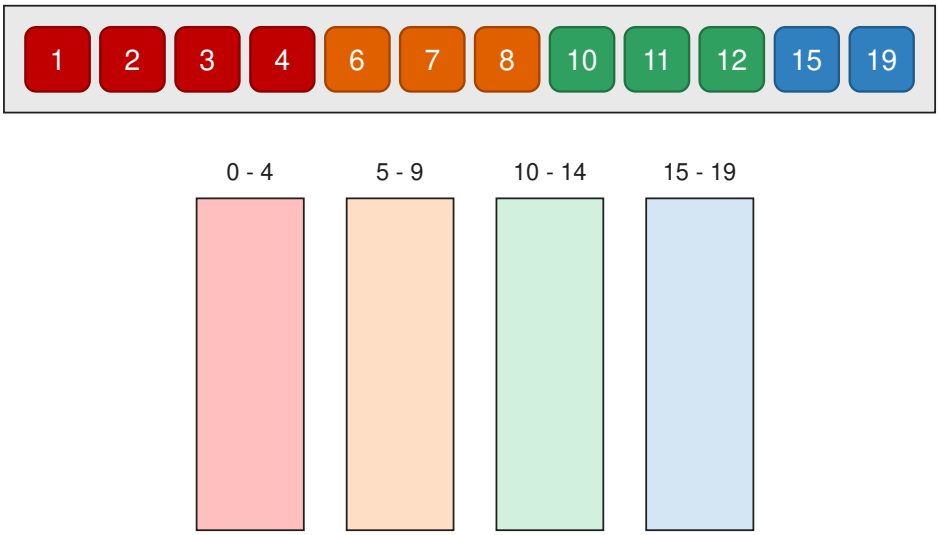
Bucket Sort: Buckets Filled



Bucket Sort: Sort Buckets



Bucket Sort: Emptied Into Array



Other Bucket Sort Variations

- Proxmap Sort
  - almost identical to the basic Bucket Sort
  - items are sorted immediately when placed in the bucket – usually an Insertion Sort
- Histogram Sort (aka Counting Sort)
  - does an initial scan of the array and creates buckets the exact size that they will be filled
  - greatly minimalizes overhead

Other Bucket Sort Variations

- Postman's Sort
  - very similar to the next sort we cover: Radix
  - sorts items by "category" of the key
- Shuffle Sort
  - array is recursively sub-divided, sorted, and merged/concatenated when complete
  - 2-bucket Shuffle Sort is essentially a Quick Sort with the pivot acting as divider between the two buckets

Bucket Summary

Bucket Sort	
Time Average	$O(n + (n^2 / b) + b)$ *where b is the # of buckets*
Time Best	$O(n)$ *when $b \approx n$*
Time Worst	$O(n^2)$ *if Insertion Sort is used*
Auxiliary space	$O(b + n)$
Stable	**Yes**
Online?	Yes (bucket filling stage only)

The 1890 Census Crisis

Computer Science to the rescue!

33

The 1890 Census Crisis

- United States Constitution:
  - population must be calculated every 10 years
  - used in the House of Representatives
- Before the 1890 Census Crisis, all this counting was done by hand…

34

The 1890 Census Crisis

- There were too many people…
  - 1880 Census barely made it within the 10-year window
  - U.S. population had continued to grow, and it could **not** be counted in 10 years
- The U.S. was still healing from the Civil War… *failing to represent each state fairly could have resulted in another war*

Herman Hollerith to the Rescue

- *Herman Hollerith* developed a machine (and concepts) that saved the U.S.
- The machine used electricity (a new idea for the time)
- Could automatically read cards and quickly, accurately tabulate results

Inventing a Solution for Sorting

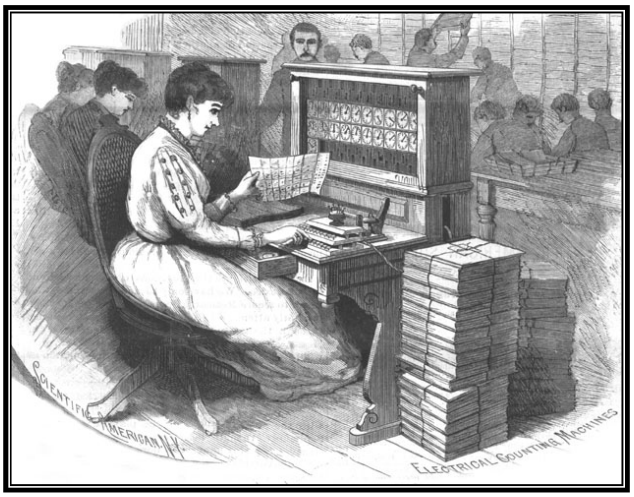
- Invented the idea to Bucket Sort on each digit of a key
- Use multiple passes starting with the 1's digit and move upwards

Herman Hollerith

- His system was used for the 1890 Census
- And, it only took **9 months**!
- Ever since, some form of tabulating machine has been used

Herman Hollerith & IBM

- The World took notice & wanted his machines
- Hollerith founded the Tabulating Machine Company
- Later it was renamed International Business Machines



Radix Sort

A dance of buckets

41

Radix Sort

- The *Radix Sort* was developed by *Herman Hollerith* in 1887 *(77 BBW)*
- The sort is completely non-comparative
- It uses a multiple Bucket Sort passes to sort data

42

Radix Sort

- Hollerith observed:
  - Bucket Sort is **stable**
  - i.e. items do not change relative positions if sorted to the same value
- He took advantage of this to sort data regardless of the size of the key

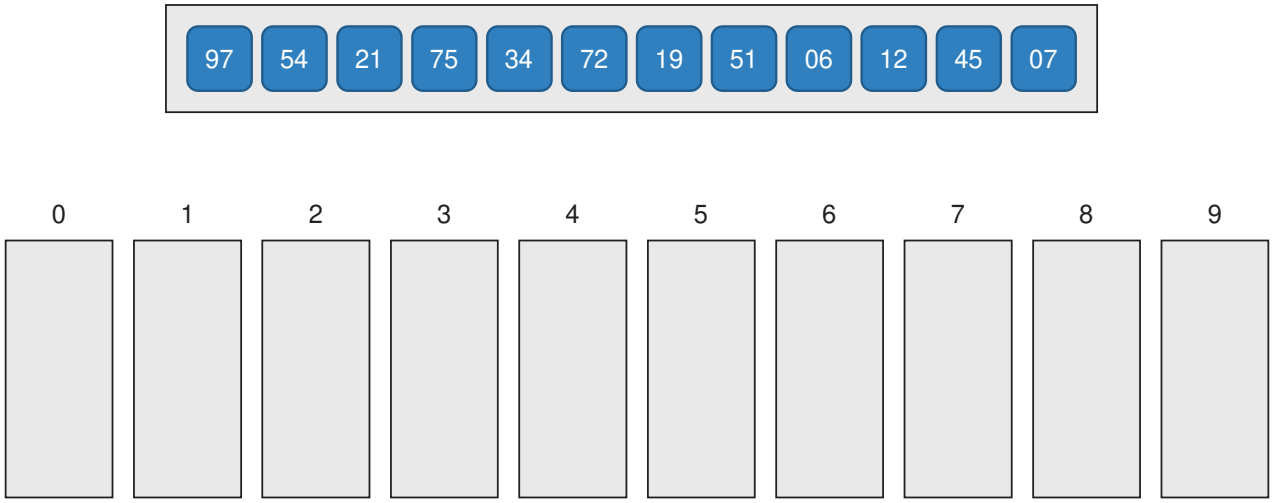
How it Works

- Radix Sort uses a Bucket Sort on each digit on the key
- This is done from the Least Significant Digit (LSD) to the most (MSD)
- After each pass, the buckets are the emptied into another set of buckets based on the next digit

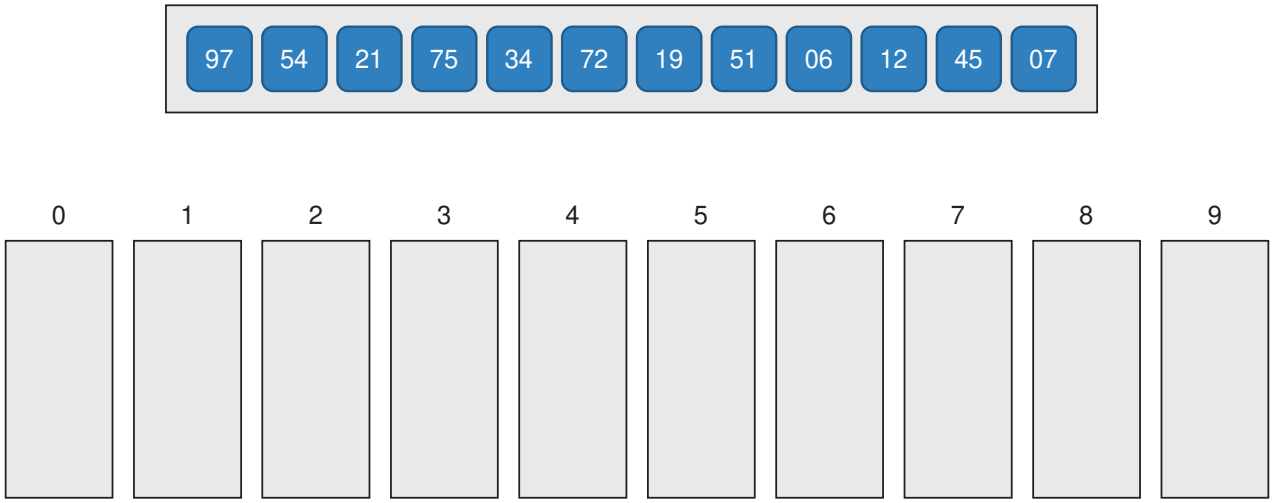
How it Works

- So, the number of buckets is equal to the number of possible digits
- Different "digits" can also be used:
  - base-10 digits for numbers (10 buckets)
  - or a single bit in the key (2 buckets)
  - or several binary bits as a group – e.g. every 4 bits for 2^4 = 16 buckets

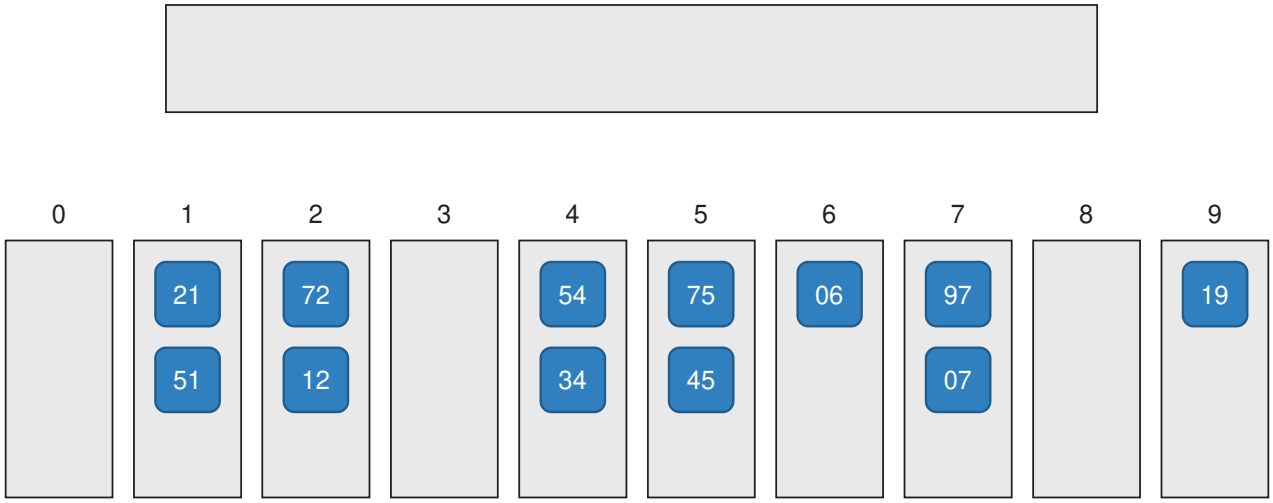
Radix Sort Example



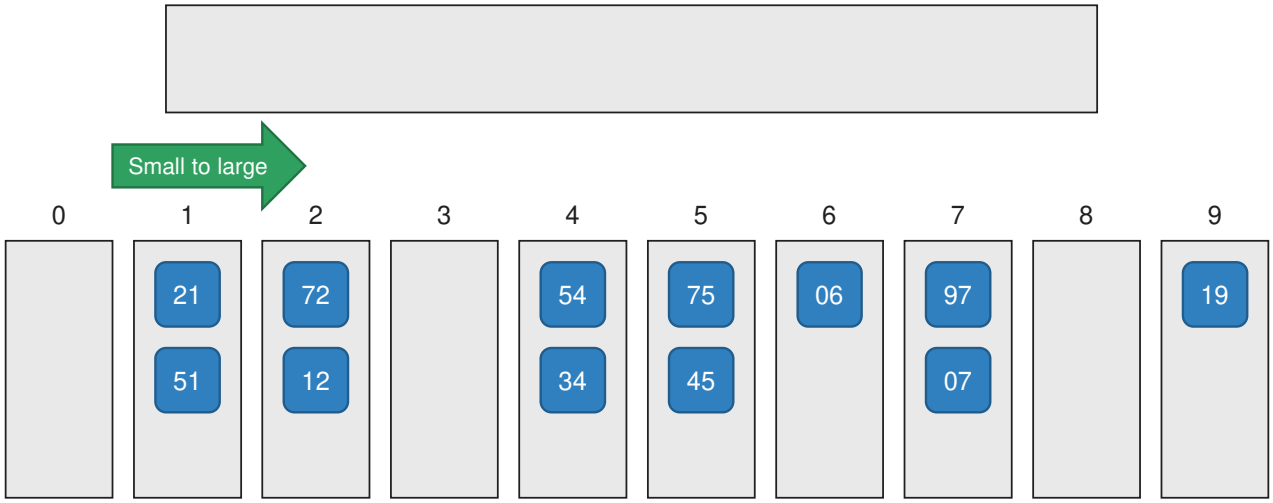
First Digit: Scatter into Buckets



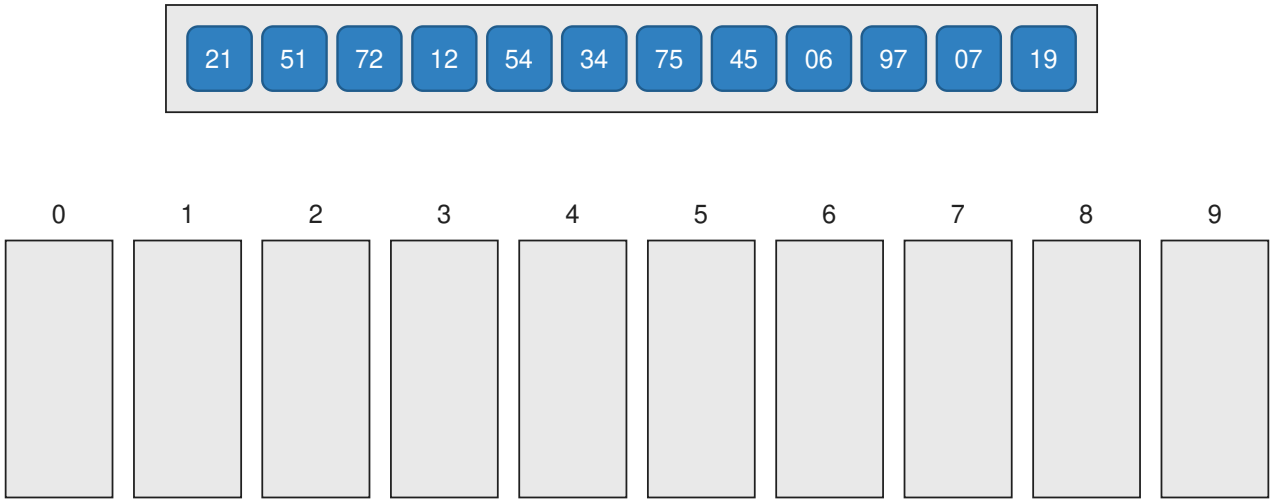
First Digit: Scattered



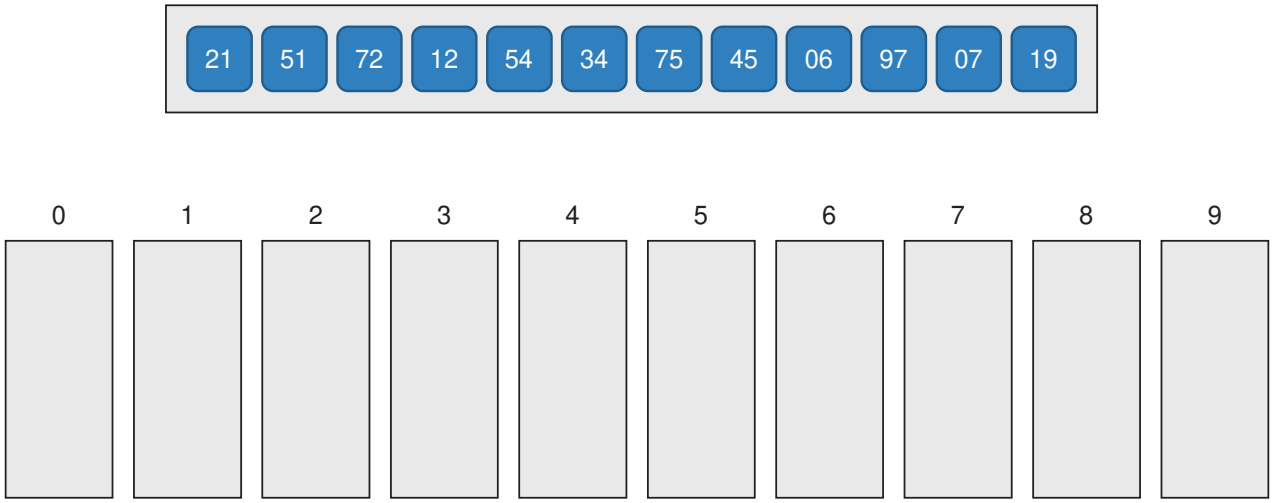
First Digit: Gather First Digit



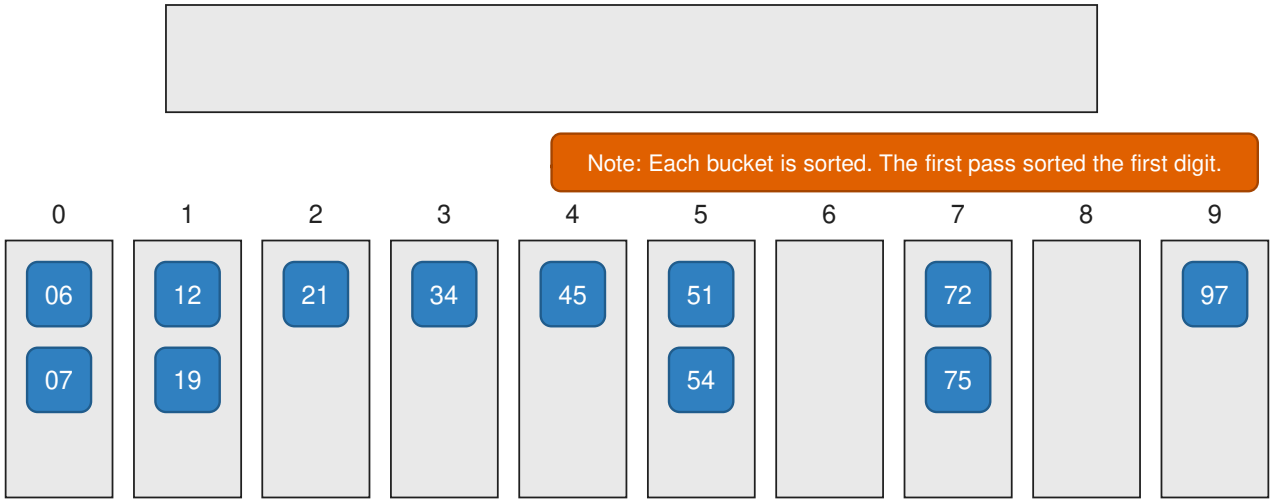
First Digit: Sorted



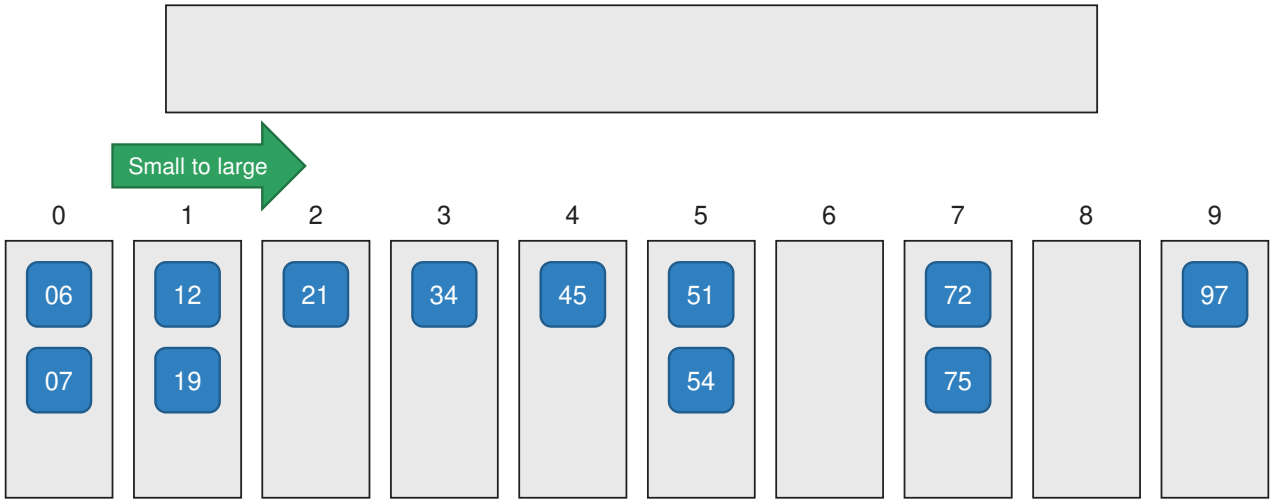
Second Digit: Scatter into Buckets



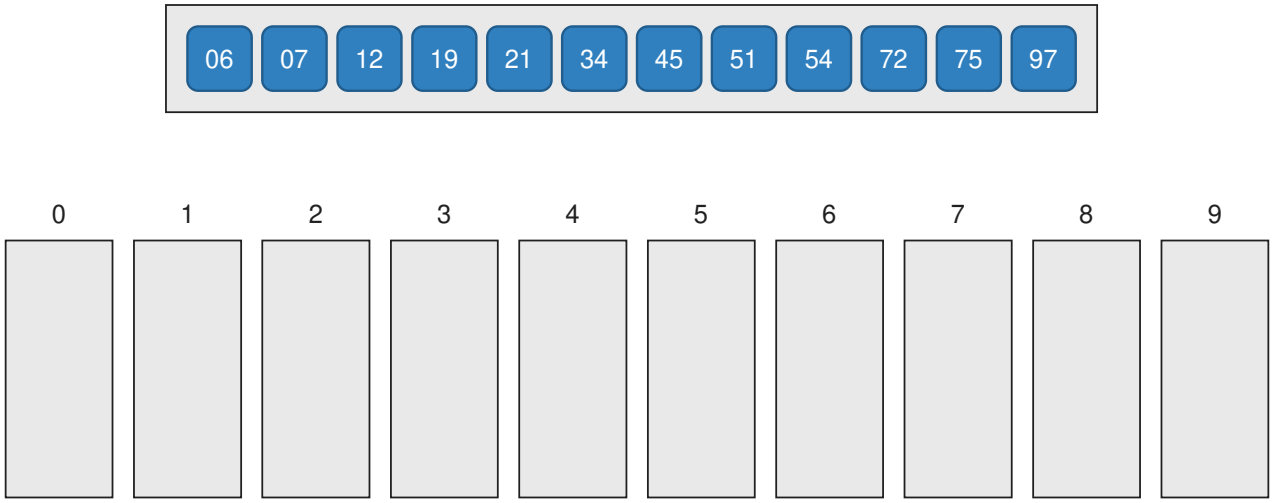
Second Digit: Scattered



Second Digit: Gather



Second Digit: Completed



Example 3: LSD Base 10

92	49	11	74	50	42	90	47	16	21

Sorted on first digit

50	90	11	21	92	42	74	16	47	49

Sorted on first 2 digits

11	16	21	42	47	49	50	74	90	92

Example 3: LSD Base 10

916	279	530	209	451	042	101	404

530	451	101	042	404	916	279	209

Sorted on first 2 digits

101	404	209	916	530	042	451	279

Sorted on first 3 digits

042	101	209	279	404	451	530	916

Time Complexity of Radix Sort

- How many passes?
  - the algorithm will pass over the array equal to the total number of digits in the key (k)
  - e.g. for social security numbers, k = 9
- So…
  - we will exam n array elements a total k number of times
  - so, it will be O(k × n)

Auxiliary Storage of Radix Sort

- How many buckets?
  - we need auxiliary storage for each array element and for a bucket for each digit
  - for a base-10 number, the number of buckets b = 10
- So…
  - we will need an extra n array elements and b buckets
  - so, it will be O(b + n)

Radix Summary

Radix Sort	
Time Average	O(k × n) *where k is the # of key digits*
Time Best	O(k × n) *actually a slow O(n)*
Time Worst	O(k × n) *actually a slow O(n)*
Auxiliary space	O(b + n)
Stable	**Yes**
Online?	No

Summary of Sorting Algorithms

Sort Algorithm	Best	Average	Worst	Aux. Storage
Bubble	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Shell	O(n log n)	$O(n^{5/4})$	$O(n^{3/2})$	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(1)
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)
Radix	O(k × n)	O(k × n)	O(k × n)	O(b + n)